

Article Generation Pipeline

Prompt builder

Retrieved passages become ranked context blocks with strict output instructions: **TITLE:** and **ARTICLE:**

Local LLM generation

OpenAI-compatible endpoint, controlled temperature, max-token budget, one repair attempt

Evaluation

ROUGE, BERTScore, title similarity, section-count difference, article-length ratio

Database upsert

generated text, reference text, and metrics are stored in Postgres

Validation layer

parse title/body, reject malformed output, repair once if needed

Runner behavior

- ▶ single-article or batch execution
- ▶ schema creation for **generated_articles**
- ▶ skip-existing logic from database state
- ▶ per-article error isolation for long runs

Cost profile

Generation dominates runtime. Retrieval, prompt assembly, and database writes are much cheaper, while larger k (the number of retrieved passages included in the prompt) and larger token budgets mainly increase latency.

LLM used (With Ollama)

Started with gemma-2-2b, then went with Phi-3.5-mini-instruct for 16K context window.

The Prompt

Write a neutral encyclopedia-style article in plain text using only the supplied passages. Use only supported facts from the passages. Do not invent details. If a detail is uncertain, conflicting, or weakly supported, leave it out. Do not mention passages, retrieval, scores, evidence, or source metadata. Do not write commentary, advice, analysis notes, a Q&A answer, or a summary about the evidence. Do not address the reader. Do not write like a review, recommendation, or box-office report. Do not output tables, lists, charts, headings, markdown,

or code fences. Write continuous prose paragraphs only. Prefer broad coverage of the most important facts over over-expanding one aspect. Avoid repeating the same fact in different wording. Higher relevance passages matter more than lower relevance passages. Use lower-scoring passages only for extra detail, not to override stronger evidence. If supported by the passages, cover subject, background, production or development, release, reception, and performance. If an aspect is not supported, omit it. This article has `{bundle.available_passage_count}`

indexed passages in total. The prompt contains `{len(passages)}` selected passages. Aim for about `{target_words}` words and about `{target_paragraphs}` paragraphs. The title must be exactly the target title. The first line must be exactly: `TITLE: <article title>` Then a blank line. Then a line that is exactly: `ARTICLE:` Then the article body in plain prose paragraphs. If you cannot follow this exact format, output nothing.

Conclusion & Future Work

Conclusion

The implemented system already provides a complete retrieval-to-generation baseline: it loads article metadata and reference text from Postgres, retrieves evidence from FAISS-indexed passages, builds a constrained prompt, generates a Wikipedia-style article draft with a local LLM, evaluates the result against the withheld reference article, and stores outputs plus metrics for reproducible comparison.

What already works well

- ▶ end-to-end reproducible pipeline
- ▶ explicit grounding in retrieved passages
- ▶ lexical, semantic, and structural evaluation
- ▶ resume-friendly batch processing via upserts

Future work

- ▶ section-wise RAG instead of single-shot generation
- ▶ better topic inference and query expansion
- ▶ stronger redundancy reduction before prompting
- ▶ tuning of k , prompt length, and model choice
- ▶ richer factuality checks beyond overlap metrics

Takeaway

The strongest next gains will likely come from better retrieval structure and more controlled generation rather than from rebuilding the pipeline itself.